AI Compiler Performance Engineering

Auto-Loops Books — ch07

Contents

1 Static Role Assignment and Schedule-Free Kernels **1**

1.1 Static roles 1

1.2 Thread mapping 4

1.3 No runtime scheduling 5

1.4 YiRage tiling 8

1.5 Key Takeaways 9

1.6 Conclusion 10

Bibliography **12**

Index **14**

Chapter 1

Static Role Assignment and Schedule-Free Kernels

Chapter 6 made on-chip residency a budgeted, checkable property; this chapter spends that budget on a specific structural choice—assigning each execution unit a fixed role at compile time so the kernel needs no runtime scheduler at all (Shah *et al.*, 2024; Mirage Project contributors, 2025). The motivation is again the launch storm: dense Llama-3.2-1B averages 847.5 GPU kernel launches per output token on H100 and OLMoE-1B/7B averages 9,305, and most of that orchestration exists only because the work-to-hardware mapping is decided dynamically, per operator, by the host (Vellaisamy *et al.*, 2026). If instead the mapping is fixed once, at compile time, the host stops dispatching and the kernel becomes "schedule-free"—a single resident program that already knows what every block and warp should do (Chen, 2026; NVIDIA, 2024). The mistake to avoid is assuming static assignment sacrifices flexibility for speed. On the contrary, fixing roles is what lets the compiler reason about residency and overlap precisely, because it knows in advance which unit produces what and when (Dao *et al.*, 2022; Wu *et al.*, 2024). This chapter develops static role assignment on GPUs and dataflow NPUs, shows how it eliminates runtime dispatch, and ends with the YiRage tiling pass that assigns roles per target.

1.1 Static roles

Static role assignment means deciding, at compile time, what each execution unit does for the entire life of the kernel, rather than letting a runtime scheduler

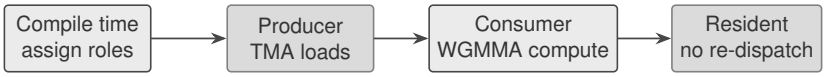


Figure 1.1: Static role assignment fixes each unit's job at compile time—producer warps move data, consumer warps compute—so no runtime scheduler is needed.

reassign work dynamically (Shah *et al.*, 2024). On a GPU the canonical instance is warp specialization: the warps of a thread block are partitioned into producer warps that only ever issue data movement and consumer warps that only ever issue computation (Shah *et al.*, 2024). FlashAttention-3 makes this concrete on Hopper—producer warps drive the Tensor Memory Accelerator (TMA) to move tiles from global to shared memory, while consumer warps drive Warpgroup Matrix-Multiply-Accumulate (WGMMA) on those tiles—and the paper notes that this separation "improves the compiler's ability to generate optimal instruction schedules" (Shah *et al.*, 2024).

Static role assignment is, in compiler terms, a form of specialization: the kernel is compiled knowing the role of each unit, so branches that would otherwise select behavior at runtime are resolved away, and each unit's instruction stream contains only the instructions for its role (Shah *et al.*, 2024). This is why the FA3 authors observe that the separation improves the compiler's ability to schedule instructions—a producer warp's stream is all loads and barrier signals, a consumer warp's stream is all matmuls and barrier waits, and neither contains the other's instructions to get in the way (Shah *et al.*, 2024). A unified, non-specialized warp would interleave both and force the instruction scheduler to reason about dependencies that the static split makes structural.

The reason a fixed **role** helps is mechanical sympathy with asynchrony. Modern GPU data movement and compute are asynchronous: TMA copies and WGMMA matmuls run on separate hardware paths and signal completion through barriers (SemiAnalysis, 2024; NVIDIA, 2022). If a single warp had to both load and compute, it would stall waiting on its own loads; by giving each warp one **static role**, the producer can run ahead filling buffers while the consumer drains them, and the two overlap naturally. This is the same double-buffering idea from Chapter 6, but expressed as a division of labor among units rather than among buffers.

Static **role** assignment is not GPU-specific; it is the native model of a dataflow NPU. On the AMD XDNA fabric each compute tile has a fixed **role** in a spatial pipeline, with its DMA engine and vector unit assigned a specific stage of the computation at compile time, and data flowing between neighboring tiles through configured streams (AMD, 2024b,a). There is no runtime that decides which tile does what—the array is configured before execution and holds that configuration.

The GPU and NPU therefore arrive at the same place from opposite directions: the GPU adds static specialization on top of a dynamically scheduled machine, while the NPU is statically scheduled by construction. This convergence is a useful design signal: when two very different architectures both reward fixing roles at compile time, it suggests the benefit is fundamental to memory-bound, low-parallelism workloads rather than an artifact of one machine's quirks (AMD, 2024a; Shah *et al.*, 2024). The book's recurring thesis—that decode is bound by movement and orchestration, not arithmetic—predicts exactly this, because static assignment attacks orchestration directly while leaving the arithmetic untouched (Vellaisamy *et al.*, 2026; Chen, 2026).

It is worth being precise about what "schedule-free" does and does not mean. It does not mean the kernel performs no scheduling—there is still an order in which a consumer warp processes its tiles. It means the *assignment* of work to units is not recomputed at runtime: no host scheduler decides which kernel runs next, and no hardware block scheduler streams fresh CTAs through the machine per operator (Shah *et al.*, 2024; Mirage Project contributors, 2025). The distinction matters because the two kinds of scheduling have very different costs. Ordering within a resident unit is nearly free; reassigning work across units, especially when it requires a host round trip, is the expensive part, and it is the part static assignment removes (Vellaisamy *et al.*, 2026).

A second clarification is why this is specifically a decode technique. During prefill there is abundant parallelism—many prompt tokens processed at once—so a dynamic scheduler has plenty of independent work to keep units busy and its overhead amortizes across a large batch (Kwon *et al.*, 2023). At batch-1 decode there is exactly one token of work per step, so any scheduling overhead is paid against a tiny amount of useful computation, and the ratio of overhead to work is at its worst (Chen, 2026; Davies *et al.*, 2025). Static role assignment is therefore not a universal speedup; it is the right structure precisely when parallelism is scarce and overhead dominates, which is the decode regime this book targets.

Taken to its limit, static **role** assignment produces a megakernel. Mirage Persistent Kernel assigns every streaming multiprocessor a **role** in an SM-level task graph and runs the entire model forward pass as one kernel, with the SMs cooperating through in-kernel scheduling rather than returning to the host between operators (Mirage Project contributors, 2025; Wu *et al.*, 2024). The **static** structure is what makes this possible: because each SM's sequence of tasks is known at compile time, there is no need to re-launch or re-dispatch, and the megakernel reduces per-token launches from thousands to one (Vellaisamy *et al.*, 2026; Chen, 2026). The lesson that carries forward is that flexibility at runtime is precisely the



Figure 1.2: The Hopper thread hierarchy; static role assignment maps work to this structure at the warp and warpgroup granularity.

thing decode cannot afford—every dynamic decision is overhead paid per token.

1.2 Thread mapping

Assigning roles requires knowing the **thread** hierarchy precisely, because roles are assigned at its granularities. On Hopper the hierarchy runs from threads, to **warp**s of 32 threads, to warpgroups of 4 contiguous **warp**s, to thread blocks (CTAs), to thread-block clusters, to the grid (Shah *et al.*, 2024). Each **thread** has at most 256 registers private to it; shared memory is addressable by all threads in a CTA; and CTAs in a cluster are co-scheduled on the same GPC with fast access to each other's shared memory (Shah *et al.*, 2024; NVIDIA, 2022). These groupings are not bookkeeping—they are the units to which roles are bound.

The warpgroup is the key granularity for decode kernels because WGMMA, the high-throughput Hopper matmul, is issued collectively by a warpgroup of four **warp**s (Shah *et al.*, 2024). Warp specialization therefore typically assigns one warpgroup the consumer **role** (issuing WGMMA) and dedicates other **warp**s to the producer **role** (issuing TMA). A subtlety that matters for residency is register reallocation: because producers issuing TMA need few registers while consumers running WGMMA need many, Hopper allows warpgroup-wide register reallocation to give the consumer warpgroup a larger share of the register file (Shah *et al.*, 2024). This is static role assignment paying off directly in the residency budget of Chapter 6—the fixed roles let the compiler skew registers toward the units that need them.

Thread mapping also determines how overlap is achieved within and across warpgroups. FlashAttention-3 overlaps the softmax computation of one warpgroup with the GEMM of another using pingpong scheduling driven by synchronization barriers, reporting an improvement from roughly 580 to 640 TFLOPS in BF16 from that inter-warpgroup overlap alone (Shah *et al.*, 2024). The reason overlap is necessary is a throughput mismatch: in the FA3 analysis the special-function exponentials of softmax take about half the cycles of the tensor-core matmul, so leaving them un-overlapped idles the tensor cores for a substantial fraction of the time (Shah *et al.*, 2024). Static role assignment is what makes the overlap

schedulable, because the compiler knows which warpgroup is doing softmax and which is doing GEMM and can place the barriers accordingly.

The cluster level adds a mapping option that did not exist before Hopper and that static assignment can exploit. Thread-block clusters group several CTAs that are co-scheduled on the same GPC and can read each other's shared memory directly through distributed shared memory, and TMA loads can be multicast to all CTAs in a cluster in one operation (NVIDIA, 2022; Shah *et al.*, 2024). For a decode kernel that means a producer role can load a weight tile once and multicast it to the consumers in several CTAs of a cluster, rather than each CTA loading its own copy—a residency win that is only schedulable because the cluster membership and the producer/consumer split are both fixed in advance. Mapping work to the cluster granularity is thus another static decision the compiler makes, trading a larger co-scheduled group for cheaper data sharing.

There is an interplay between role mapping and the occupancy budget of Chapter 6 that the compiler must resolve jointly. Dedicating warps to a producer role consumes warp slots that could otherwise run consumers, so the split is not free: too many producers and the consumers starve for compute slots, too few and the consumers stall waiting on data (Shah *et al.*, 2024; NVIDIA, 2022). The right ratio depends on the throughput mismatch between data movement and compute for the specific kernel, which is exactly the kind of target- and shape-specific decision that argues for a compiler pass over a hand-tuned constant. At batch-1 decode, where the matmul is a thin matrix-vector product, the balance often tilts toward fewer compute warps and more attention to keeping them fed, the opposite of a throughput-shaped prefill kernel (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

On a CPU or NPU the mapping granularities differ but the principle holds. A CPU maps roles to **thread**s and SIMD lanes, pinning a decode **thread** to a core and a NUMA node as Chapter 6 described; an NPU maps roles to tiles in its array, with each tile a fixed stage (Goodfellow *et al.*, 2016; AMD, 2024a). In every case the mapping is chosen so that the units that must communicate are close in the hierarchy—warps in a CTA sharing shared memory, tiles adjacent in the array sharing a stream—so that the data a role produces reaches the role that consumes it without leaving fast memory (SemiAnalysis, 2024; AMD, 2024b).

1.3 No runtime scheduling

The payoff of static roles is that the kernel needs no **runtime** scheduler, which is where the decode latency is won. In a conventional stack the host decides, per operator, which kernel to launch and on what data, paying a **dispatch** cost for

Table 1.1: Where work-to-unit **dispatch** happens, and the per-token cost it implies for batch-1 decode (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024).

Model	Dispatch decided	Per-token launch cost
Per-operator kernels	Host, at runtime	Thousands of launches
Persistent + tile scheduler	In-kernel, at runtime	Fixed CTA count
Static megakernel (MPK)	Compile time	One launch

every one of the thousands of launches a token requires (Vellaisamy *et al.*, 2026). A schedule-free kernel moves that decision to compile time: the work-to-unit mapping is baked in, so the only **runtime** activity is the computation itself (Mirage Project contributors, 2025; Shah *et al.*, 2024).

The persistent kernel is the GPU mechanism that eliminates **runtime dispatch** for the multi-tile case. Instead of launching one CTA per work tile and relying on the hardware's block scheduler to stream them, a persistent kernel launches a fixed number of CTAs equal to the SM count—132 CTAs on an H100 SXM5, one per SM—and each CTA loops over a fraction of the work tiles, assigned by an in-kernel tile scheduler (Shah *et al.*, 2024). This decouples physical CTAs from logical work, so the kernel can overlap the epilogue of the current tile with the prologue loads of the next, and avoids re-incurring launch and prologue cost for every tile (Shah *et al.*, 2024). The tile scheduler is a small piece of in-kernel logic, not a host round trip, so the **dispatch** decision never leaves the GPU.

Mirage Persistent Kernel takes the elimination of **runtime dispatch** to the whole model. Its SM-level task graph is built at compile time, and an in-kernel parallel **runtime** executes the tasks with decentralized scheduling across SMs, so the entire forward pass runs as one launch with no host involvement between operators (Mirage Project contributors, 2025). The reported 1.2–6.7× end-to-end latency reduction over kernel-per-operator serving is, in large part, the cost of **runtime dispatch** that the static structure removed (Mirage Project contributors, 2025; Chen, 2026). The same principle is the NPU's default: because the array is configured before execution, there is no **runtime** that decides tile assignment, and the per-token cost is whatever the static schedule fixed (AMD, 2024b,a).

The tile scheduler deserves a closer look because it is where the small amount of necessary dynamism lives. In a persistent kernel the scheduler is in-kernel logic that hands the next work tile to whichever resident CTA is free, which both balances load and lets the kernel decouple the number of physical CTAs from the number of logical tiles (Shah *et al.*, 2024). CUTLASS generalizes this with schemes like Stream-K, where the scheduler can split a tile's reduction across CTAs and fall

back to simpler data-parallel or Split-K modes by heuristic, and on Hopper it constructs the schedule in units of clusters rather than single tiles (Shah *et al.*, 2024). The important property for decode is that all of this stays on the GPU: the scheduler is a few instructions of in-kernel arithmetic, not a host callback, so it removes launch overhead without reintroducing dispatch latency.

Contrast this with the NPU, where even the in-kernel scheduler disappears. Because the XDNA array is configured before execution and each tile holds a fixed stage, there is no per-token assignment decision at all—the "schedule" is the physical configuration of the fabric, and running a token is just clocking data through it (AMD, 2024b,a). This is the most extreme point on the static-to-dynamic axis: the GPU per-operator stack recomputes assignment thousands of times per token, the persistent kernel computes it in-kernel a fixed number of times, the megakernel computes it once at compile time, and the static NPU never computes it at runtime at all (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025). Each step down that axis trades flexibility for fewer per-token decisions, and decode rewards moving as far down it as the workload's regularity allows.

There is an honest limit to schedule-free kernels. A fully static assignment can load-balance poorly when work is irregular—the persistent tile scheduler exists precisely to redistribute uneven work, such as the variable row counts under causal masking, while staying in-kernel (Shah *et al.*, 2024). The design judgment is how much dynamism to keep: enough to balance genuinely irregular work, but not so much that the host re-enters the loop. For batch-1 decode, where the per-token work is regular and small, the answer is to keep almost nothing dynamic, which is why decode is the workload where schedule-free kernels pay off most (Kwon *et al.*, 2023; Davies *et al.*, 2025).

A practical consequence for serving teams is that the win compounds with model depth. Each transformer layer that a per-operator stack executes adds its own quota of launches and dispatch decisions, so a deep model pays the orchestration tax once per layer per token; a static megakernel pays it once per token regardless of depth (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025). This is why the reported megakernel speedups are largest for the deepest, most launch-dense models—the very OLMoE-style mixture-of-experts configurations that issue 9,305 launches per token in the TaxBreak measurement (Vellaisamy *et al.*, 2026). The static structure converts a cost that scaled with the number of operators into a constant, which is the kind of asymptotic improvement that a faster inner loop can never deliver (Chen, 2026).

1.4 YiRage tiling

The book's reference compiler turns static role assignment into a per-target pass. **YiRage** carries a **tiling** pass that, given a fused region and a target, chooses tile shapes and assigns each execution unit its static role—which warps are producers and consumers on a GPU, which tiles are which stage on an NPU—and emits the corresponding schedule-free kernel (Chen and YiRage Contributors, 2026). Because the residency budget of Chapter 6 and the role assignment of this chapter are decided together, the **tiling** pass can guarantee that the chosen tiles fit the on-chip memory of the target while the role assignment keeps producers and consumers overlapped (Wu *et al.*, 2024; Shah *et al.*, 2024).

The **tiling** decision is genuinely different per target, which is why it must be a pass and not a fixed template. On a GPU the pass picks a tile size that fits the 228 KB shared-memory budget, assigns a consumer warpgroup for WGMMA and producer warps for TMA, and reallocates registers toward the consumer (NVIDIA, 2022; Shah *et al.*, 2024). On an XDNA NPU the same logical computation is tiled to fit the 64 KB tile SRAM, with double-buffered DMA and a fixed tile-to-stage assignment (AMD, 2024a,b). The math is identical; the **tiling** and role assignment are not, and a plan transferred blindly from one to the other is either illegal or wasteful (SemiAnalysis, 2024).

The pass must also decide how far to fuse before assigning roles, because the fusion boundary and the role split are coupled. Fuse too little and the kernel is back to per-operator launches with their dispatch tax; fuse too much and the combined working set blows the residency budget or the register file, forcing a spill that negates the win (NVIDIA, 2022; Chen, 2026). The **tiling** pass resolves this by growing the fused region while continuously checking the residency metadata, stopping at the largest region whose static role assignment still fits the target's fast memory (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). This is the same stopping rule Chapter 6 stated for fusion in general, now applied with the role assignment in hand: the boundary sits where one more fused operator would force either a spill or a producer/consumer imbalance that starves the tensor cores. Because the check is per target, the boundary lands in a different place on an H100 than on an XDNA tile, which is exactly why a single hand-tuned kernel cannot be the answer across a heterogeneous fleet (AMD, 2024a; Davies *et al.*, 2025).

The pass treats tile-shape selection as a constrained search rather than a lookup. For a GPU consumer the tile must be large enough to amortize the WGMMA setup and keep the tensor cores busy, small enough that the producer can keep its double buffers filled within the shared-memory budget, and shaped so the access pattern

is bank-conflict-free—three constraints that interact, so the pass explores shapes rather than applying a fixed one (Shah *et al.*, 2024; NVIDIA, 2022). For an NPU tile the dominant constraint is simply fitting the double-buffered operands in the 64 KB local SRAM, after which the pass picks the largest tile that maximizes reuse (AMD, 2024a,b). The output of the search is not just a number but a complete static plan: tile shape, producer/consumer warp split, register allocation, and the barrier placement that realizes the overlap.

This is also where the **tiling** pass interacts with quantization and precision. A lower-precision tile—FP8 or int8—moves fewer bytes and fits more elements per buffer, which loosens the residency constraint and can change the optimal tile shape and even the producer/consumer ratio (Shah *et al.*, 2024; Goodfellow *et al.*, 2016). The pass therefore cannot choose tiles independently of the numeric format; it must co-decide them, because the format determines how much of the working set is resident and therefore how the roles should be balanced. For a memory-bound decode kernel this coupling is the whole game: the format choice and the **tiling** together set bytes/token, and the role assignment sets whether the movement of those bytes overlaps the compute (Vellaisamy *et al.*, 2026; Chen, 2026).

What makes the **YiRage tiling** pass composable with the rest of the book's machinery is that it consumes the same residency metadata the memory pass produces and feeds the megakernel search of Chapter 20. The superoptimizer proposes a fusion; the memory pass certifies its footprint fits; the **tiling** pass assigns static roles and tiles that realize the fusion as a schedule-free kernel on the target (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025). The result is that "static role assignment" stops being a hand-coding technique a CUDA expert applies and becomes a compiler decision that is made, checked, and re-made per target automatically.

1.5 Key Takeaways

- **Fix roles at compile time to remove runtime scheduling.** Assigning each unit a static role—producer or consumer on a GPU, a fixed stage on an NPU—lets the kernel run without a host scheduler, removing the per-token dispatch that dominates batch-1 (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026). Flexibility at runtime is overhead paid per token (Chen, 2026).
- **Warp specialization overlaps data movement and compute.** Producer warps issuing TMA and consumer warpgroups issuing WGMMA overlap

naturally, and pingpong scheduling overlapped softmax with GEMM for 580→640 TFLOPS in FlashAttention-3 (Shah *et al.*, 2024). Static roles are what make the overlap schedulable (SemiAnalysis, 2024).

- **Map roles to the right hierarchy granularity.** Roles bind to warps and warpgroups on a GPU (WGMMA is warpgroup-collective), threads and SIMD lanes on a CPU, and tiles on an NPU; co-locate communicating roles so produced data reaches its consumer on chip (Shah *et al.*, 2024; AMD, 2024a). Register reallocation skews the file toward consumers (NVIDIA, 2022).

- **Persistent kernels keep dispatch in-kernel.** Launching CTAs equal to the SM count (132 on H100 SXM5) with an in-kernel tile scheduler eliminates per-tile launch overhead and overlaps epilogue with prologue (Shah *et al.*, 2024). MPK extends this to one launch for the whole model, reporting 1.2–6.7× end-to-end latency reduction over kernel-per-operator serving (Mirage Project contributors, 2025).

- **Make role assignment a per-target compiler pass.** The YiRage tiling pass chooses tiles and assigns static roles per target, composing with the residency memory pass and megakernel search (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). The same math gets a different, checked schedule on each class, and the fusion boundary lands wherever the target's residency budget allows (AMD, 2024b).

1.6 Conclusion

Static role assignment is the structural move that turns Chapter 6's residency budget into a schedule-free kernel: fix what each warp, warpgroup, or tile does at compile time, and the runtime scheduler—and its per-token dispatch cost—disappears (Shah *et al.*, 2024; Mirage Project contributors, 2025). Warp specialization and persistent kernels show the GPU path, the XDNA array shows the natively static NPU path, and MPK shows the limit case of one launch per token; in every case the win is that dynamism removed at compile time is overhead not paid at runtime (Vellaisamy *et al.*, 2026; Chen, 2026). The YiRage tiling pass makes the assignment a per-target compiler decision rather than a hand-coding feat, choosing tile shapes and role splits that fit each target's residency budget automatically (Chen and YiRage Contributors, 2026). Static roles, however, only pay off if the data each role needs arrives on time, which is the subject of Chapter **??**: TMA double-buffer pipelines and the asynchronous movement that keeps producer and consumer roles

fed, so the statically assigned consumers never stall waiting on data (SemiAnalysis, 2024; Dao *et al.*, 2022).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 2, 3, 5, 6, 7, 8, 9, 10

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 2, 5, 6, 7, 8, 9, 10

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 3, 6, 7, 8, 9, 10

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 8, 9, 10

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 11

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 3, 7, 8

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 5, 9

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 3, 5, 7

Mirage Project contributors (2025). Mirage persistent kernel (MPK): A compiler and runtime for mega-kernelizing tensor programs. `https://github.com/mirage-project/mirage`. Transforms LLM inference into a single megakernel; SM-level task graph (tGraph); cross-operator pipelining; 1.2–6.7× latency reduction. 1, 3, 6, 7, 9, 10

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 2, 4, 5, 8, 9, 10

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 2, 5, 8, 10, 11

Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. (2024). FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*. Hopper warp specialization (producer TMA / consumer WGMMA), persistent kernels with tile scheduler, pingpong scheduling. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 3, 5, 6, 7, 9, 10

Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, M. K., Velliengiri, P., Miao, X., Padon, O., and Jia, Z. (2024). Mirage: A multi-level superoptimizer for tensor programs. *arXiv preprint arXiv:2405.05751*. μGraph kernel/block/thread levels; joint algebraic+schedule+custom-kernel superoptimization; abstraction-based pruning; probabilistic equivalence via PIT over finite fields (LAX); 1.1–2.9×. 1, 3, 8, 9, 10

Index

Persistent kernel, 1

Static role assignment, 1

Warp specialization, 1